

CS2106 Cheatsheet, AY26/27 S1

Zhao Wu and Tien Cheng

September 15, 2026

1 Introduction

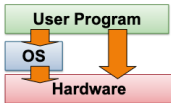
What is OS?

OS is a program that acts as an **intermediary** between a **computer user** and the **computer hardware**. (Simplified definition)

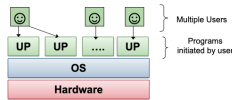
- E.g.
 - Windows, macOS, Linux, Solaris, FreeBSD
 - iOS, Android
 - PS5, Xbox, Nintendo Switch
 - Smart TV, Smart Watch

Brief History of OS

- First computer:
 - OS Type:** No OS
 - Program directly interacts with hardware.
 - Reprogram by physically changing the hardware configuration (cables, switches, punched paper tape)
 - Advantage:**
 - Minimal overhead due to OS
 - Disadvantage:**
 - Not portable (to port, need to manually rewrite the program)
 - Inefficient use of the computer
 - E.g. Electronic Numerical Integrator And Computer (ENIAC), Harvard Mark I
- Mainframes
 - Characteristics:**
 - No interactive interface (program via paper tape, magnetic tape, punch card)
 - Batch Processing Only
 - OS Type:** Batch OS
 - User still interacts with hardware directly
 - Additional information for OS (e.g. resource required, job specification)
 - Advantage:**
 - Execute user programs (i.e. jobs) one at a time
 - Disadvantage:**
 - Simple batch processing is inefficient as the CPU is idle during I/O.
 - Multiprogramming is absent.
 - E.g. IBM 360



- Time-Sharing OS
 - OS Type:** Time-Sharing OS
 - Advantage:**
 - Allow multiple users to interact with the machine using terminals (teletypes)
 - User job scheduling (illusion of concurrency)
 - OS manages CPU time, memory and storage
 - Virtualization of hardware (each program executes as if it has all the resources to itself)
 - E.g. Apple II PC, IBM PC



- Personal OS
 - Windows model:
 - Single user at a time, but possibly more than 1 user can access
 - General time-sharing model
 - Unix model:
 - One user at the workstation, but other users can access remotely

- General time-sharing model

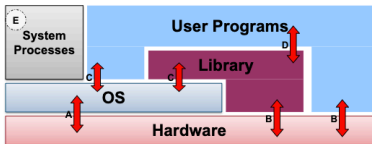
1.1 Motivation of OS

- Abstraction**
 - Hide low-level details and present higher-level functionality to the user
 - Motivation:
 - Large variation in hardware configuration, but the same hardware has well-defined functionality. (E.g. hard disk rotation speed varies, but it can store and retrieve information)
 - Efficiency, Programmability and Portability
- Resource Allocator**
 - Manage all resources (CPU, Memory, I/O) and arbitrate potentially conflicting requests for efficient and fair resource use
 - Motivation:
 - Program requires multiple hardware resources to run.
 - Multiple programs should run simultaneously for better utilisation
- Control Program**
 - Control execution of programs to:
 - Prevent errors and improper use of the computer
 - Motivation:
 - Program may "misuse" the computer
 - Bugs (accident), Virus, Malware (malicious)
 - Multiple users can share the computer
 - Security, isolation and protection

1.2 OS Structures

High-Level View of OS

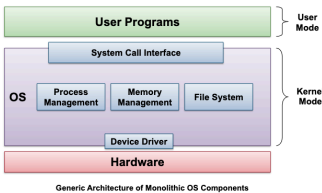
- OS is software that runs in **Kernel Mode** (i.e. direct access to all hardware resources)
 - Can't use system calls in kernel code
 - Can't use normal libraries and I/O
- Other software operates in **User Mode** (i.e. limited/controlled access to hardware resources)



- A:** OS executing machine instructions
- B:** Normal machine instructions executed (program/library code)
- C:** Calling OS using **system call interface**
- D:** User program calls library code
- E:** System processes
 - Provide high-level services, usually part of OS

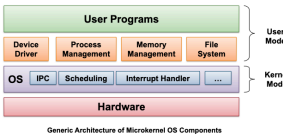
OS Structures

- Monolithic**
 - Big program (e.g. Linux source code)
 - If Kernel fails, BSOD
 - Better performance



- Microkernel**

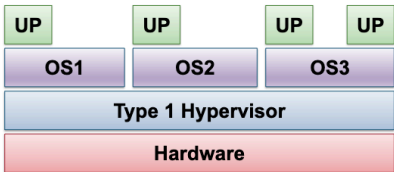
- Smaller and cleaner abstraction
- Provides basic and essential facilities:
 - Inter-Process Communication
 - Address space management
 - Thread management
- Higher-level OS services are run outside of the kernel, using IPC to communicate.
- Lower performance



1.3 Virtual Machines

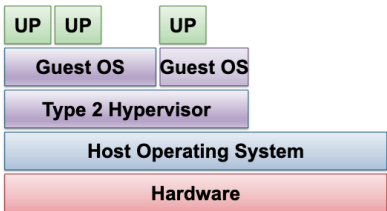
A **Virtual Machine** emulates the underlying hardware and is created and managed by a **Hypervisor** (aka **Virtual Machine Monitor (VMM)**)

- Type 1 Hypervisor



- Type 1 hypervisor:**
 - AKA bare-metal hypervisor
 - Provides individual **virtual machines** to guest OSES
 - E.g., IBM VM/370, VMware ESXi

- Type 2 Hypervisor



- Type 2 hypervisor**
 - Runs in host OS
 - Guest OS runs inside Virtual Machine
 - e.g., VMware Workstation, VirtualBox, QEMU

2 Process Abstraction

Motivation

- Allowing only one program to run at a time is inefficient, so the OS enables multiple programs to run concurrently.
- To switch between programs, the OS needs to save the context of the current program and load the context of the next program.
- A **process** (or task or job) is the OS abstraction for a running program.

2.1 Process Context

To allow multiple programs to run concurrently, the OS manages the following components:

Layer	Component	Stores
Memory	Text	program instructions
	Data	global variables and static variables
	Stack	collection of stack frames (calls + locals)
	Heap	region of memory used to store dynamically allocated data
Hardware	General-Purpose Registers (GPRs)	temporary data
	Program Counter (PC)	next instruction address
	Stack Pointer (SP)	top of stack frame address
	Frame Pointer (FP)	fixed location in stack frame address
OS	Process ID (PID)	unique process ID
	Process State	current process state

2.2 Stack Memory & Function Calls

Stack Memory

- The **stack** is a region of memory used to support function calls.
- Each function invocation **pushes a new stack frame onto the stack**, while each function return **pops the top stack frame**.
- Each stack frame contains (CS2106 convention, caller-pushed first):

Component	Who	Why
Local variables	Callee	Callee's own local variables
Parameters (only those that don't fit in registers)	Caller	so the callee can read the parameters of function call
Saved GPRs	Callee	Copy of GPRs which callee modifies; protect the caller's values across the call
Saved old SP	Callee	restore the caller's SP
Saved old FP	Callee	restore the caller's FP
Return address / Saved PC	Caller	return to the right instruction after the callee returns

- The stack pointer (SP) points to the top of the stack
- The frame pointer (FP) points to the base of the current stack frame
- We use a frame pointer because the stack pointer tracks the current top of the stack, which can move as local variables are pushed and popped. The FP is fixed, so we can use a fixed offset to access arguments and local variables.
- Restoring the SP does not erase the stack frame, so always initialise local variables as uninitialised local variables can contain leftover values from previous calls.

Function Call Convention

Function calling convention differs depending on the hardware and software. The following is an example of such convention in 2106:

- Function Call Preparation:**
 - Caller: pass parameters using registers and/or the stack
 - Caller: save return program counter onto the stack
- Transfer of Control from Caller to Callee:**
 - Callee: **save registers used by the callee, save the old frame pointer**, save the old stack pointer
 - Callee: allocate space for local variables of the callee on the stack
 - Callee: adjust the stack pointer to point to the new stack top, **adjust the frame pointer**
- Callee Function Executes** (during that time, the callee may invoke other functions, but this can be abstracted away)
- Callee Function Returns**
 - Callee: place the return result in a register (if applicable, usually a dedicated return register)

2. Callee: **restore saved registers and frame pointer**, restore the saved stack pointer

5. **Transfer of Control from Callee back to Caller using saved PC:**

1. Caller: uses the return result (if applicable)

2. Caller: continues execution of the program

```
void f(int a, int b)
{
    int c;

    a = 123;
    b = 456;
    c = g(a, b);
    ...
}

int g(int i, int j)
{
    int a;

    a = i + j;
    return a * 2;
}
```

local var "a"

123

456

Saved SP

Return PC

Stack Frame for f()

New SP

Old SP

Parameters

Register Spilling

- Register Spilling:** When a function has more arguments than the number of registers, the extra arguments are spilled to the stack
- Even if neither function is short of registers, the caller and callee may use the same registers, so we need to save the registers and restore them after the call.
- There are two methods to save the registers:
 - Callee-saved:** the callee saves the old values of the registers into its stack frame and restores them after the call
 - Caller-saved:** the caller saves needed registers into its stack frame and restores them after the call
- CS2106 convention: all registers are callee-saved unless otherwise specified

2.3 Heap Memory

Heap Memory

- The **heap** is a region of memory used to store dynamically allocated data.
- Dynamically allocated data in C: `malloc` and `free`
- Dynamically allocated data cannot be stored in:
 - Data region: size must be known at compile time
 - Stack: the data may outlast the function call
- Heap management is harder than stack management as allocation and deallocation can happen in arbitrary orders, leading to fragmentation.

2.4 Process States

Process State Model

New

Ready

Running

Blocked

Terminated

create

admit

switch: scheduled

switch: release CPU

event occurs

event wait

exit

- A process can be in one of the following states:
 - New:** process has been created but not fully initialised/admitted to the system
 - Ready:** process is waiting to execute
 - Running:** process is currently executing
 - Blocked:** process is waiting for an event to occur (e.g. I/O completion, signal)
 - Terminated:** process has finished execution

Multi-Process Management

- With one CPU core, at most one process can be running at a time
- With m CPU cores, at most m processes can be running at a time
- CS2106 convention: assume one CPU core unless otherwise specified
- Different processes may be in different states at the same time

Process Queues

admit

ready queue

switch by scheduler

Running Process

exit

release cpu

blocked queue

event wait

CPU

- The OS maintains a queue of processes for each state
 - Ready Queue:** processes that are ready to be scheduled to run on the CPU
 - Blocked Queue:** processes that are waiting for an event to occur
- More than one process can be in the ready and blocked queues at the same time
- There may be separate queues for different types of blocked processes (e.g. I/O blocked, signal blocked)

2.5 Process Control Block (PCB)

A Process Control Block (PCB) or Process Table Entry is a data structure that describes the execution context for a process, maintained by the Kernel.

PCB₁

PCB₂

PCB₃

.....

Process Table

PC, FP, SP,

GPRs

Memory Region Info

PID

Process State

Process Control Block

Text

Data

Heap

Stack

Memory Space of a Process

2.6 System Calls

A System Call is an API to the OS that allows a user program to request services from the Kernel.

Difference in OS

- Unix Variant:
 - Follows POSIX standards
 - Small number of calls
- Windows Variant:
 - Uses Win API across different Windows versions
 - New version of windows add more calls.
 - Huge number of calls

System Call Mechanism

- User invokes the library call (e.g. `getpid()`)
- Library call places the system call number into a register.
- Library call invokes the **trap** instruction to switch from user mode to kernel mode.
- In kernel mode, the **dispatcher** identifies the system call and passes control to the appropriate system call handler.
- System call handler executes the system call

6. System call handler ended by restoring CPU state and return to user mode.

7. Library call returns the result to the user program.

PCB₁

PCB₂

PCB₃

.....

Process Table

PC, FP, SP,

GPRs

Memory Region Info

PID

Process State

Process Control Block

Text

Data

Heap

Stack

Memory Space of a Process

Exception and Interrupt

- Exception:**
 - Occurs during the execution of a program
 - Synchronous in nature (i.e. caused by the program itself)
 - E.g. divide by zero error or a page fault
- Interrupt:**
 - Occurs during the execution of a program
 - Asynchronous in nature (i.e. caused by external events)
 - E.g. hardware interrupt or a software interrupt

void f()
{
 ...
 Statement S1;
 ...
}

void handler()
{
 1. Save Register/CPU state
 2. Perform the handler routine
 3. Restore Register/CPU
 4. Return from interrupt
}

1. Exception/Interrupt occurs:

Control transfer to a handler routine automatically

2. Return from handler routine:

Program execution resume

May behave as if nothing happened

2.7 A Case Study of Processes in Unix

Process Abstraction

Unix process management centres on `fork()`, `exec()`, `exit()`, and `wait()`.

In Unix, an entry in the PCB consists of:

- Identification:
 - PID: Process ID (integer identifier)
- Information:
 - Process State: Running, Sleeping/Suspended, Stopped, Zombie, etc.
 - Parent PID (PPID)
 - Cumulative CPU time
 - Other accounting and resource-management information

Use `ps` (process status) to inspect process information; `man ps` for options.

2.7.1 Process Creation: Fork, Exec

Unix vs Windows

- Windows:** spawn a new process in (pretty much) a single syscall — `path`, arguments, etc.
- Unix:** Two-step approach:
 - Parent calls `fork()` clones the current process (parent continues from the instruction after `fork()`)

2. Child calls `exec()` to replaces the current process (child discards old text/data/stack and execution state).

fork()

`int fork()` is the primary Unix mechanism for creating a process. It creates a child by duplicating currently executing parent.

- Both parent and child continue from **immediately after** the `fork()` call.
- Child is initially an **almost exact duplicate**:
 - Same code and **initial** address-space **contents** (conceptually duplicated, **not** shared)
 - Copied register and execution context
 - Different PID and PPID
 - Different `fork()` return value: parent receives the child's PID; child receives 0

```
printf("I am ONE\n");
fork(); // second process spawned; both continue from here
printf("I am seeing DOUBLE\n");
// I am ONE
// I am seeing DOUBLE
// I am seeing DOUBLE
```

Using fork()'s return value

Parent and child execute the same instructions after `fork()`. Use the return value to assign distinct work:

- `result != 0` — parent (can continue accepting work)
- `result == 0` — child (performs a separate task)

```
int result = fork();
if (result != 0) { // parent
    printf("P:My Id is %i\n", getpid());
    printf("P:Child Id is %i\n", result);
} else { // child
    printf("C:My Id is %i\n", getpid());
    printf("C:Parent Id is %i\n", getpid());
}
// P:My Id is 1234 / P:Child Id is 5678
// C:My Id is 5678 / C:Parent Id is 1234
```

Nondeterministic scheduling

- Parent/child order is **nondeterministic**: parent first, child first, or interleaved.
- On a single core they do not execute at the exact same instant, but the scheduler may alternate.

Independent Address Spaces

Parent and child initially see **equivalent values** after `fork()`, but do **not** share ordinary memory.

- Stack, heap, data, and code image are **conceptually duplicated**.
- Modifying a variable in one process does **not** modify the other.
- Open files and working directory are inherited (shared kernel resources), not the address space.
- If var starts as 1234, parent can increment its copy while child decrements its own. Print order may vary; each process's private value changes independently.

exec() replaces the process image

By itself, `fork()` only duplicates the current program. `exec()` **replaces** the current process image:

- Replaces current code and data with a new executable
- Begins at the new program's entry point
- Discards the old stack and execution state
- Retains the same PID** and broader process identity
- Variants: `execL`, `execv`, `execve`, `execpL`, `execvp`
- Successful `exec()` does not return** — the old image no longer exists

Command-Line Arguments

```
int main(int argc, char *argv[])
{
    argc: number of arguments, including the program name
    argv: array of C strings; argv[0] is conventionally the executable name
}
```

execL()

`execL(path, arg0, ..., NULL)` supplies the executable path, the argument list, and a final NULL.

```
execL("/bin/ls", "ls", "-al", NULL); // replaces current program with ls -al
// NULL marks the end of the argument list
```

init and Process Tree

A process can only be created by forking an existing process, so Unix processes form a **process tree**.

- Root is **init**: created by the kernel during boot, traditionally **PID 1**
- Common ancestor of user processes; typically spawns OS/system programs

- Adopts orphaned processes
- Implementation differs by OS; on many Unix systems `init` is a symlink (e.g. to `systemd`)
- Cannot be killed** even though it runs in user space; if it crashes → **kernel panic**

2.7.2 Process Termination & Lifecycle

exit()

exit(int status) terminates the current process. **Does not return.**

- `exit(0)`: normal/successful termination
- Non-zero: error or abnormal condition
- Returning from `main()` **implicitly** invokes `exit()`; `main`'s return value becomes the exit status
- Open output streams are flushed; file descriptors are released
- On process `exit()`:
 - Process state is set to **Zombie**
 - Most resources are released (e.g. file descriptors)
 - Some information is **not releasable** (so the parent can `wait()`):
 - PID
 - Exit status
 - Process accounting (e.g. CPU time)

wait()

wait(int *status) lets a parent synchronise with any child(s):

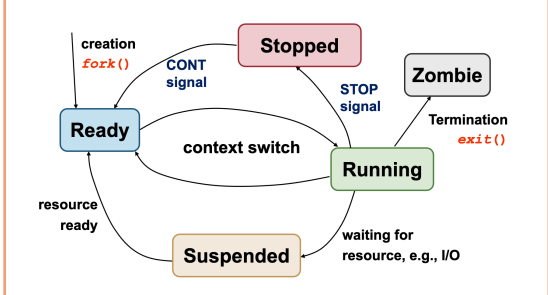
- Blocks** until at least one child terminates
- Returns the PID of a terminated child
- Stores the child's exit status through `status`, unless `NULL`
- Kernel can write into the parent's memory because it is privileged
- Variants:
 - `waitpid()` — wait for a specific child
 - `waitid()` — wait for child state changes

Zombie vs Orphan

- Zombie**: child that has **exited** but has not yet been **reaped** by its parent via `wait()`
 - Cannot execute or be meaningfully killed (already dead)
 - Remaining PCB occupies a process-table entry; too many can exhaust the table (older Unix: may need reboot)
- Orphan**: a **still-running** child whose parent has terminated
 - `init` becomes its pseudo-parent
 - When the adopted child later terminates, `init wait()`s and cleans up
 - If the parent dies while the child is **already a zombie**, `init` reaps that leftover state too

Parent-Child Lifetime

- Parent forks a child
- Child optionally execs a new program
- Child exits and becomes a zombie
- Parent waits
- Kernel removes the child's remaining process-table entry



Unix Process States

Running, Sleeping/Suspended, Stopped, Zombie. Major transitions:

- `fork()` creates a **ready** process
- Context switch: `ready` → `running`
- Running waiting for a resource (e.g. I/O) → `suspended`
- Resource ready → `suspended` → `ready`

- Stop/continue signals move between running/stopped and ready
- `exit()` → zombie, then final cleanup after `wait()`

3 Process Scheduling

Concurrent processes refers to processes that progress in execution at the same time. This can be achieved by:

- Virtual Parallelism: Pseudo-parallelism
- Physical Parallelism: Multiple CPU / Core to allow parallel execution

Timeslicing

Interleaving of instruction between processes is called **timeslicing**

- Context switching operation is required before one process can handover to another process.
- Can be done in 1-Core CPU (timesliced for instructions) or multiprocessor (timesliced for instructions & CPU core)

The diagram shows two processes, A and B, executing on a single CPU. Process A has segments a1, a2, a3, a4, a5, a6. Process B has segments b1, b2, b3, b4, b5. Context switching operations occur at time points t1, t2, and t3, where the CPU switches from one process to the other.

3.1 Scheduling Algorithms

Scheduling Problem

When there are more ready-to-run processes than available CPUs, **Scheduler** decides process should be chosen to run based on **Scheduling Algorithm**

Scheduling Algorithm is tailored based on process behaviour and process environment.

- Process Behavior**: a typical process cycles between
 - CPU-activity: Compute-bound spends majority of time
 - IO-activity: IO-bound spends majority of time
- Processing Environment**:
 - Batch Processing: No interaction required, no need to be responsive.
 - Interactive (or Multiprogramming): Active user interacts with system. Should be responsive.
 - Real time processing: Have deadline to meet, usually periodic process.

For all processing environments:

- Fairness**: Processes should get a fair share of CPU time and no starvation
- Utilization**: All parts of computing system should utilized

Scheduling Algorithms:

- Non-preemptive (Cooperative)**: Process stayed scheduled (in running state) until it blocks or give up the CPU voluntarily.
- Preemptive**: Process is given a fixed time quota to run, and at the end of the time quota, the process is suspended.

3.1.1 Scheduling for Batch Processing

Batched processing system has no user interaction and mostly dominated by non-preemptive scheduling.

Key criterion:

- Turnaround time: Total time taken (finish time - arrival time)
- Throughput: number of tasks finished per unit time
- CPU Utilization: Percentage of time when CPU is working on a task

First Come First Serve (FCFS)

- Tasks are stored on First-In-First-Out (FIFO) queue.
- First task in the queue to run until task is done or task is blocked.
- Blocked task is removed from the FIFO queue, and placed at the back of queue when it is ready.

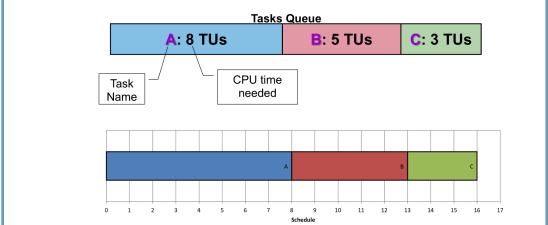
Characteristics:

- Non-preemptive algorithms

- No Starvation

Shortcomings:

- Convoy Effect: CPU-bound and IO-bound processes cannot be run concurrently.
- Reordering of task can reduce average waiting time.



■ The average total waiting time for 3 tasks

□ $(0 + 8 + 13)/3 = 7$ Time Units

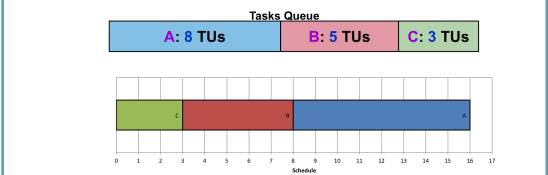
Shortest Job First (SJF)

Mechanism:

- Select the task with the smallest total CPU time

Characteristics:

- Non-preemptive algorithm
- Need to know total CPU time for a task in advance
- Starvation is possible as algorithm is biased towards short jobs
- Guarantees smallest average waiting time as task with shortest job always comes first



■ The average total waiting time for 3 tasks

□ $(0 + 3 + 8)/3 = 3.66$ Time Units

Predicting CPU Time

- Guess the future CPU time requirement by previous CPU-Bound phases

$$\text{Predicted}_{n+1} = \alpha \text{Actual}_n + (1 - \alpha) \text{Predicted}_n$$

- Actual_n : Most recent CPU time consumed
- Predicted_n : Past history of CPU time consumed
- α : Weight placed on recent event
- Predicted_{n+1} : Latest prediction

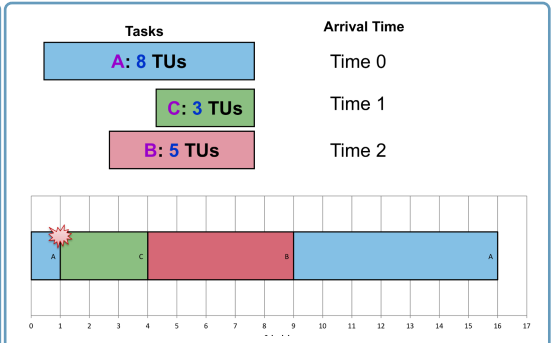
Shortest Remaining Time (SRT)

Mechanism:

- Select job with shortest remaining (or expected) time

Characteristics:

- Preemptive algorithm
- When a new shorter job arrives, will stop current job and switch to shorter job



3.1.2 Scheduling for Interactive Environment

Interactive Environment uses mostly preemptive algorithm for good response time with scheduler that runs periodically.

Key Criterion:

- Response time
- Predictability

Timer & Time Quantum

- Interval of Timer Interrupt (ITI)**: time it takes to interrupt and invoking the OS scheduler, typically 1ms-10ms.
- Time Quantum**: Execution duration given to a process, that must be multiples of timer interrupt. Time Quantum could be constant or variable among the processes, typically 5ms-100ms.

The diagram shows a Gantt chart with tasks A, B, and C. Blue arrows indicate 'Scheduler Triggers' (timer interrupts) occurring every 10ms. Red arrows indicate 'Context Switches' where the scheduler switches between tasks. The Time Quantum is 20ms.

Round Robin (RR)

Mechanism:

- Tasks are stored on First-In-First-Out (FIFO) queue.
- First task in the queue to run until a fixed time quantum elapsed, or task is done, or task is blocked.
- Task is then placed at end of queue for another turn
- Blocked task is removed from the FIFO queue, and placed at the back of queue when it is ready.

Characteristics:

- Preemptive algorithm (of FCFS)
- Response time guarantee: Time taken before a task get CPU is bounded by $(n - 1)q$ where n is number of tasks and q is quantum
- Timer interrupt needed for scheduler to check on quantum expiry
- Choice of time quantum duration matters:
 - Big quantum is higher CPU utilization but longer waiting time
 - Small quantum is bigger overhead but shorter waiting time

Execution Timeline

No other task in queue

Quantum for A expires

Task B and C arrived

A Blocks on I/O

C give up CPU

Mechanism:

- Prioritise task with higher priority value.

Variants:

- Preemptive version
 - Higher priority process can preempt (override) running process with lower priority
- Non-preemptive version
 - Late coming high priority process has to wait for next round of scheduling

Shortcomings:

- Low priority process may starve
- Possible solution:
 - Decrease the priority of currently running process after every time quantum
 - Give the current running process a time quantum and ensure its not considered in next round of scheduling

Tasks	Arrival Time	Priority (1=highest)
A: 8 TUs	Time 0	3
C: 3 TUs	Time 1	1
B: 5 TUs	Time 1	5

Multi-Level Feedback Queue (MLFQ)

Mechanism:

- If Priority(A) > Priority(B): run A.
- If Priority(A) == Priority(B): A and B runs in RR

Characteristics:

- Minimizes both response time for IO bound and turnaround time for CPU bound processes.

- 3 Queues: Q2 (highest priority), Q1, Q0
- A single long running job
 - Try to apply the rules and check your understanding

Lottery Scheduling

Mechanism:

- Give out "Lottery Tickets" to different system resources (CPU Time, I/O devices)
- When a scheduling decision is needed:
 - A lottery ticket is randomly chosen among eligible Tickets to grant the resources
 - In long run, a process holding X% of tickets can win X% of lottery held and use the resource X% of the time

4 Inter-Process Communication

Inter-Process Communication mechanisms is needed for transferring of information between processes.

4.1 Shared-Memory

A shared memory region M is created, which will be attached by the processes P_1 and P_2 , enabling communication.

Advantages:

- Efficient: OS only needs to Create and Attach memory region
- Ease of use: information of any type or size can be written easily in shared memory space

Disadvantage:

- Synchronization between resources is harder
- Implementation is usually harder

- OS is involved in step 1 and 2 only

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/shm.h>

int main() {
    int shmid, *shm;

    // Use shmget syscall to create a shared memory in
    // IPC_PRIVATE: Create a new, private segment that only related
    // processes (e.g. parent/child after fork) can share by passing around
    // this shmid.
    // Create the segment if it doesn't exist, with Unix permissions 0600
    // owner can read and write, nobody else can.
    shmid = shmget(IPC_PRIVATE, 3 * sizeof(int), IPC_CREAT | 0600 );
    // Exit if Memory cannot be created
    if (shmid == -1) exit(1);

    // Attach memory by initializing the pointer
    // NULL: Let kernel choose free virtual address
    // 0: no extra flag, default read/write access
    shm = (int*) shmat(shmid, NULL, 0);
    // Exit if memory cannot be attached
    if (shm == (int*) -1) exit(1);

    // Declare value not ready;
    shm[0] = 0;
    // Sleep while the value is ready
    while(shm[0] == 0) {
        sleep(3);
    }

    for (int i=0; i<3; i++) {
        printf("Read %d from shared memory. \n", shm[i+1]);
    }

    // Detach and destroy shared memory region
    shmdt( (char*) shm);
    shmctl(shmid, IPC_RMID, 0);
}
```

```
return 0;
}
```

4.2 Message Passing

Process P_1 prepare messages M and send to P_2 using system calls, where M is stored in kernel memory space. Requires extra properties like Name and Synchronization.

Advantages:

- Portable: can be implemented on diff processing environment
- Easier Synchronization: when sync primitives is used, sender and receiver are implicitly synchronized

Disadvantage:

- Inefficient: Requires OS intervention
- Extra Copying

- The Msg have to be stored in kernel memory space
- Every send/receive operations need to go through OS (i.e., a system call)

Direct Communication

Sender and Receiver of message explicitly name the other party (ie. unix domain socket). One-to-one communication between processes.

Indirect Communication

Messages are sent / received from message storage known as mailbox or port (i.e. unix message queue). Many-to-Many communication between processes.

Blocking Primitives (synchronous)

Receiver is blocked until message has arrived.

Non-Blocking Primitives (asynchronous)

Receiver either receive the message if available or some indication that message is not ready.

4.2.1 Unix Pipes

In Unix, a process has 3 default communication channels: stdin (scanf), stdout (printf), stderr. Declared using `"|"`.

Pipe functions as:

- Circular bounded byte buffer: Writers wait when buffer is full
- Implicit Synchronization: Readers wait when buffer is empty

Depending on Unix version, pipes may be:

- Half-duplex: unidirectional with one write end and one read end
- Full-duplex: bidirectional with any end for read and write

```
#define READ_END 0
#define WRITE_END 1

int main()
{
    int pipeFd[2], pid, len;
    char buf[100], *str = "Hello There!";

    pipe( pipeFd );

    if ((pid = fork()) > 0) { /* parent */
        close(pipeFd[READ_END]);
        write(pipeFd[WRITE_END], str, strlen(str)+1);
        close(pipeFd[WRITE_END]);
    } else { /* child */
        close(pipeFd[WRITE_END]);
        len = read(pipeFd[READ_END], buf, sizeof(buf));
        printf("Proc %d read: %s\n", pid, buf);
        close(pipeFd[READ_END]);
    }
}
```

4.2.2 Unix Signal

Quick form of inter-process communication, sent to a process using an asynchronous notification regarding an event.

- Eg. Kill, Interrupt, Stop, Continue, Memory Error, Arithmetic Error...

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void myOwnHandler( int signo )
{
    if (signo == SIGSEGV){
        printf("Memory access blows up!\n");
        exit(1);
    }
}

int main()
{
    int *ip = NULL;

    if (signal(SIGSEGV, myOwnHandler) == SIG_ERR)
        printf("Failed to register handler\n");

    *ip = 123;

    return 0;
}
```